

Reliable Completion in AI Agents

Research statement

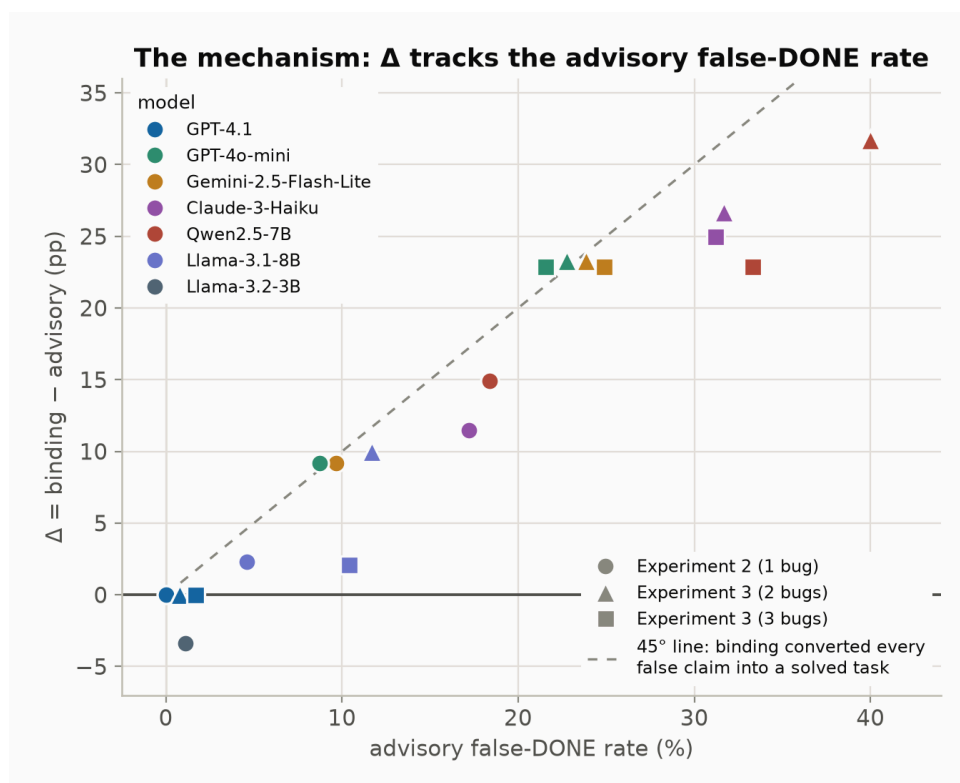
Kushagra Bhatnagar · bhatnagar.kushagra.m@u.nus.edu

Coding agents are graded on whether their final output is correct, and this work studies an earlier event, the moment an agent reports that the work is finished. A model can pass every test it can see and still be wrong, because part of the specification was never shown to it. My experiments hold the tasks, the model and the feedback fixed, and vary only who decides the task is over and what the agent can see when that decision is made. Across four preregistered experiments the wrong claims concentrated almost entirely at one identifiable state.

Reliable agents need the surrounding system to track which requirements remain unmet and to refuse a completion claim that nothing supports, because greater model capability makes the claim more frequent rather than more careful.

The experiments

An earlier programme attached an external checker to a repair loop in two ways, advisory, where it only reported what it found, and binding, where its pass was required before the episode could end. On 87 frozen bug-fix tasks the binding version lifted gpt-4o-mini from 79 of 87 solved to 87 of 87, a gain of 9.2 points at $p = 0.0078$, and left gpt-4.1 at 87 of 87 in both arms. Every one of the eight failures binding removed was a wrong completion claim rather than a repair the model could not perform, which raised the question this work studies, whether the model lacked information or the authority to stop.

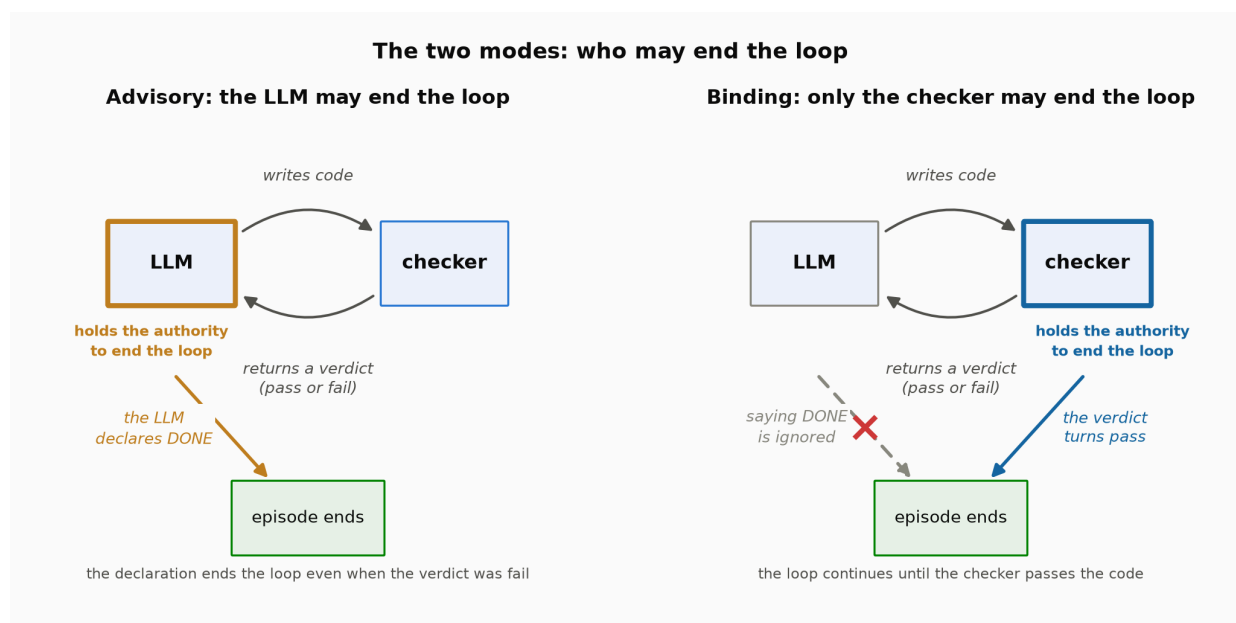


From the earlier programme, the gain from binding against how often the model claimed falsely under advisory, one point per model and difficulty. The gain tracks that rate, which is why a short probe that counts false claims can tell a designer in advance whether gating will pay.

Testing that needed tasks where the hidden information is real rather than an artifact of a vague prompt. GAUNTLET is 452 broken Python functions across seven difficulty levels, frozen with a hash, whose correct behaviour depends on a convention the task never states, drawn from a catalogue of

93 attested readings across 41 domains. At the harder levels the tests arrive in stages, so a model that passes everything visible still faces requirements it has not been shown.

claude-haiku-4.5, claude-sonnet-5 and claude-opus-5 then played 68 tasks under two rules, one where a completion claim ends the episode and one where the harness ignores it. Willingness to claim rose with capability, from 37 of 68 tasks for haiku to 57 for sonnet and 67 for opus, while wrong claims went from 0 to 2 and then 4, so opus claimed almost everywhere and was wrong most often. In 5 of the 6 wrong claims every failing test belonged to a stage the model had never seen. Removing the claim channel rescued 7 of 96 tasks in the hard band against 1 of 107 in the easy one, so a gate is worth most where the model could still have fixed the error.



The two ways the earlier programme could end an episode. Under advisory the model's own DONE ends the loop even when the last verdict was a failure, and under binding only the checker's pass ends it, so the claim carries no authority.

The obvious explanation is that nothing tells the model how many stages a task has. A second corpus of 159 tasks pinned mechanical difficulty flat, and its first version leaked the task class at a classifier accuracy of 0.9097 against a base rate of 0.5000, which I documented and rebuilt until the classifier fell below its base rate. On the rebuilt corpus claude-opus-5 played every arm of the three experiments that followed. Adding the exact hidden-stage count to the contract then changed nothing, because 12 of 115 paired tasks flipped with six going each way at $p = 0.6128$, and the warned model still claimed 69 times and was wrong 19 of them. It could nonetheless tell which tasks were risky, claiming on 97 of 117 visits to tasks that hide nothing against 23 of 121 on tasks that hide a convention.

A stated reward for stopping early also failed to move the claim rate, and five of the six flips went against the prediction at $p = 0.9844$, which rules out a large increase and licenses nothing further. What did move it was one line of arithmetic, since printing how many stages remain in the feedback at the moment of the decision, rather than promising it in the instructions at the start, dropped claiming from 44 of 72 tasks to 31 of 71, with 13 of the 14 flips going that way at $p = 0.0009$, and wrong claims from 12 to 6. Running the control twice against itself put the churn floor at 2 of 71 tasks, so the counter's 14 flips are seven times the noise, and the flips in the two nulls were movement without direction rather than silence.

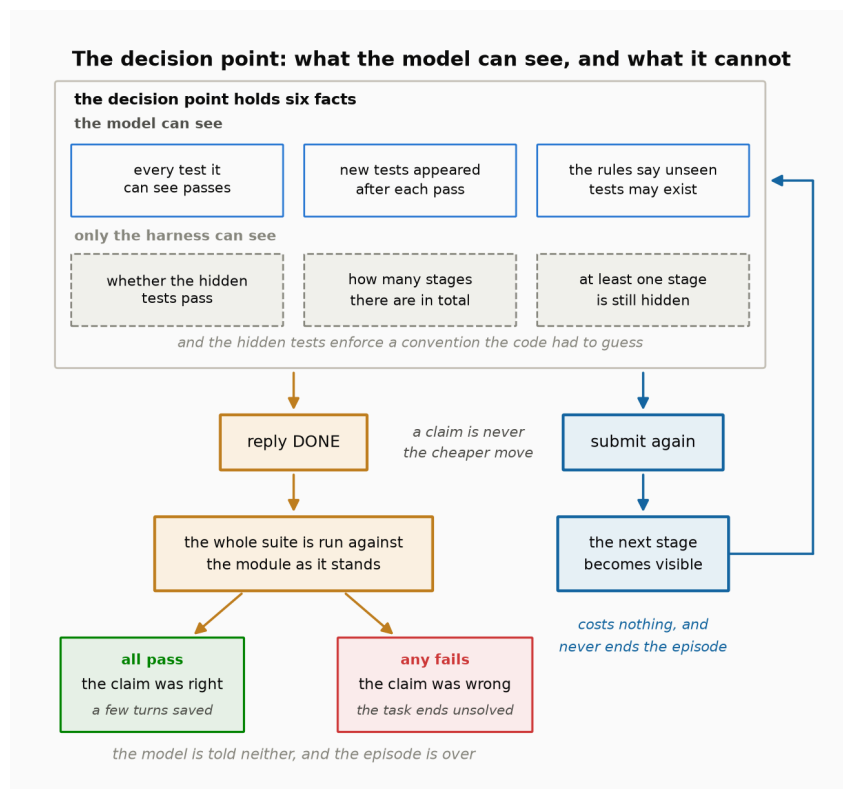
The wrong claims

Across the programme the models said DONE 608 times in 856 live episodes and were wrong 74 times, and 73 of those happened at the same state, where every visible test passes and at least one stage of hidden tests remains. Once every stage had been released the models claimed 344 times and none was wrong, and across 115 turns where a test the model could see was failing they claimed

Paper · Who Decides When the Task Is Done? · <https://doi.org/10.5281/zenodo.21698264>

Code and data · <https://github.com/kushagrab21/binding-feedback-experiment>

nothing at all. The model appears to hold the information it needs without using it at the decision, because showing it a count it had already been given is the only intervention that moved the behaviour.



The state where 73 of the 74 wrong claims were made. Every test the model can see passes, the harness alone knows a stage is still hidden, and submitting again costs nothing and never ends the episode, so a claim from here is never the cheaper move.

Current work

Locating this failure inside a model requires an open-weights model that shows it, so I registered a screening gate and ran Qwen2.5-Coder-7B-Instruct, Qwen2.5-Coder-32B-Instruct and Qwen2.5-72B-Instruct over the 24 tasks where the Anthropic models made wrong claims. Arrivals at the decision state rose from 4 to 58 and then 74 while claims at that state stayed at zero, so the failure did not transfer at these sizes and the negative is written up at full volume. Every wrong claim in this programme came from an Anthropic model and none of the three open-weights models claimed at the decision state at all, so the behaviour has appeared only in that family, and why they differ is untested. The earlier programme found wrong claims across seven models from five providers, so this is a limit of the screen rather than evidence that one company's models are alone in failing. One size in the middle was never run and the largest model comes from a different family, so the three do not license a claim about size. Screening continues on a stronger open model, and the forward-pass work waits until one passes the gate, because probing a model without the behaviour would measure nothing.

Method

I register every phase before spending money on it, and the tasks are frozen and hashed before the first episode runs. The two arms of a comparison differ in one respect only, and every number regenerates from committed files. Of 14 registered predictions, 8 were confirmed, 5 refuted and 1 unresolved, and I write up the refuted ones as fully as the rest. The work so far is 11 live runs and 856 episodes across two corpora, at a total cost of \$61.25. A manuscript is in preparation.

Previous work

The engineering before this reached the same conclusion from the other direction. I built three Python packages that turn written rules into programs which run step by step and let anyone replay a run to check its result, and a data analysis agent that runs its own analysis steps and logs every action for review. Two whitepapers set out the same argument earlier. Blockchain of Reasoning gives each step of a computation a fingerprint so the chain can be replayed and checked by someone else, and the RAM architecture puts provenance and reproducibility around the same loop. All of it needed explicit state and an outside check before what it returned could be trusted, which is the claim the experiments now test directly.

Open questions

Four questions follow. The first is whether the counter worked through its content or through the presence of one more sentence, which needs a placebo arm that is designed but not yet run. The second is whether a model that shows the failure represents the unfinished work internally, and whether changing that representation changes the claim. The third is whether an agent can be required to hand over evidence that the work is done rather than a bare claim. The fourth is whether checking strictness can be chosen per task and model rather than fixed everywhere, since the value of a gate concentrates where the model can fix its errors but stops early.

Paper · Where a Coding Agent Decides It Is Finished? · <https://zenodo.org/records/21917996>

Code and data · <https://github.com/kushagrab21/gauntlet>

Research overview · <https://github.com/kushagrab21>